Multi-Vehicle Navigation

Preface: In multi-robot autonomous cooperative operation scenarios, multiple vehicles need to independently perform self-localization, costmap maintenance, and local dynamic obstacle avoidance planning under a shared global map. This course uses two vehicles (`robot1` and `robot2`) as an example. Through namespaces, the sensor topics, TF tree, and Nav2 navigation stack are isolated independently, demonstrating the complete workflow in which the two vehicles share one global map and navigate independently to designated goal points.

---

1. Program Description

This feature assigns different navigation goal points to the two vehicles in RViz2. Based on the unified global map and their respective AMCL Monte Carlo localization, the two vehicles autonomously plan globally optimal paths. During driving, they use LiDAR to detect unknown obstacles and the other vehicle in real time, avoid them, and finally reach their designated goals safely.

1.1 Functional and Network Requirements

1. **Same Local Area Network Configuration**: Both vehicles and the control terminal (virtual machine/PC) running RViz must be on the same Wi-Fi LAN;
2. **Same ROS_DOMAIN_ID**: The `ROS_DOMAIN_ID` of the primary vehicle, follower vehicles, and the control computer must be consistent (the default is usually 61);

1.2 Map File Preparation

Multi-vehicle navigation relies on a pre-built 2D grid map (including the `.yaml` description file and the `.pgm` image file). The map files must be placed in the following default path on the control terminal or the vehicles:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pgm
```

> [!IMPORTANT]
> **Map Service Note**: The global map service (`map_server`) only needs to be **started once** on the virtual machine or control computer. Broadcasting the global `/map` topic makes it available to all vehicles on the LAN; the two vehicles themselves do not need to start a map node again.

---

2. Program Startup Steps

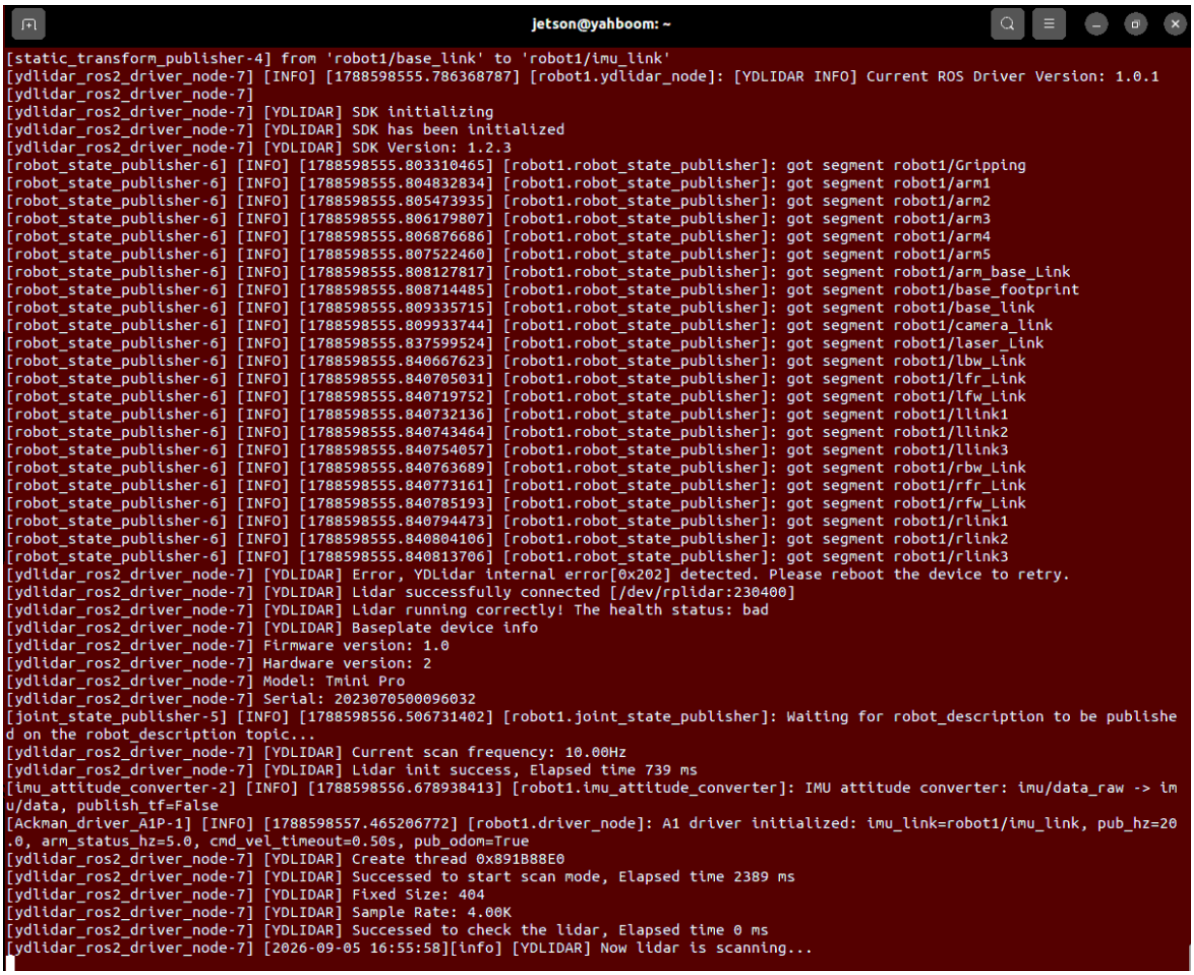
It is recommended to place the two vehicles in the approximate starting area corresponding to the map. To ensure smooth navigation computation, the map service, visualization, and multi-vehicle AMCL/Nav2 nodes are recommended to run on the control computer (or virtual machine) configured with multi-device communication.

2.1 Step 1: Start the Chassis and LiDAR on Both Vehicles

Run the following command in the terminal of **robot1**:

```
ros2 launch yahboomcar_multi A1p_laser_bringup_multi_launch.xml
robot_name:=robot1
```

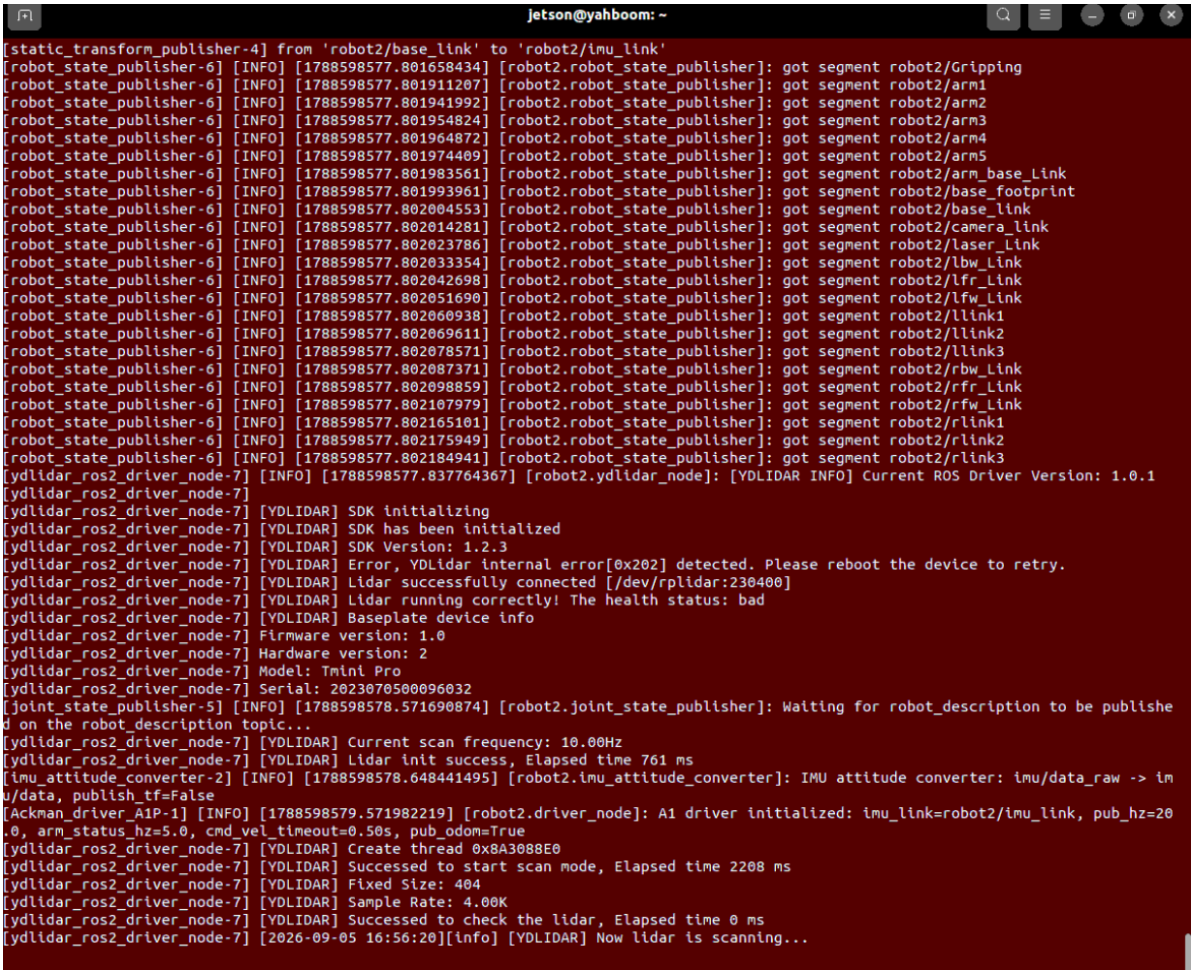
After successful startup, the terminal prints the LiDAR and chassis EKF filtering status:



Run the following command in the terminal of **robot2**:

```
ros2 launch yahboomcar_multi A1p_laser_bringup_multi_launch.xml
robot_name:=robot2
```

Startup success status:



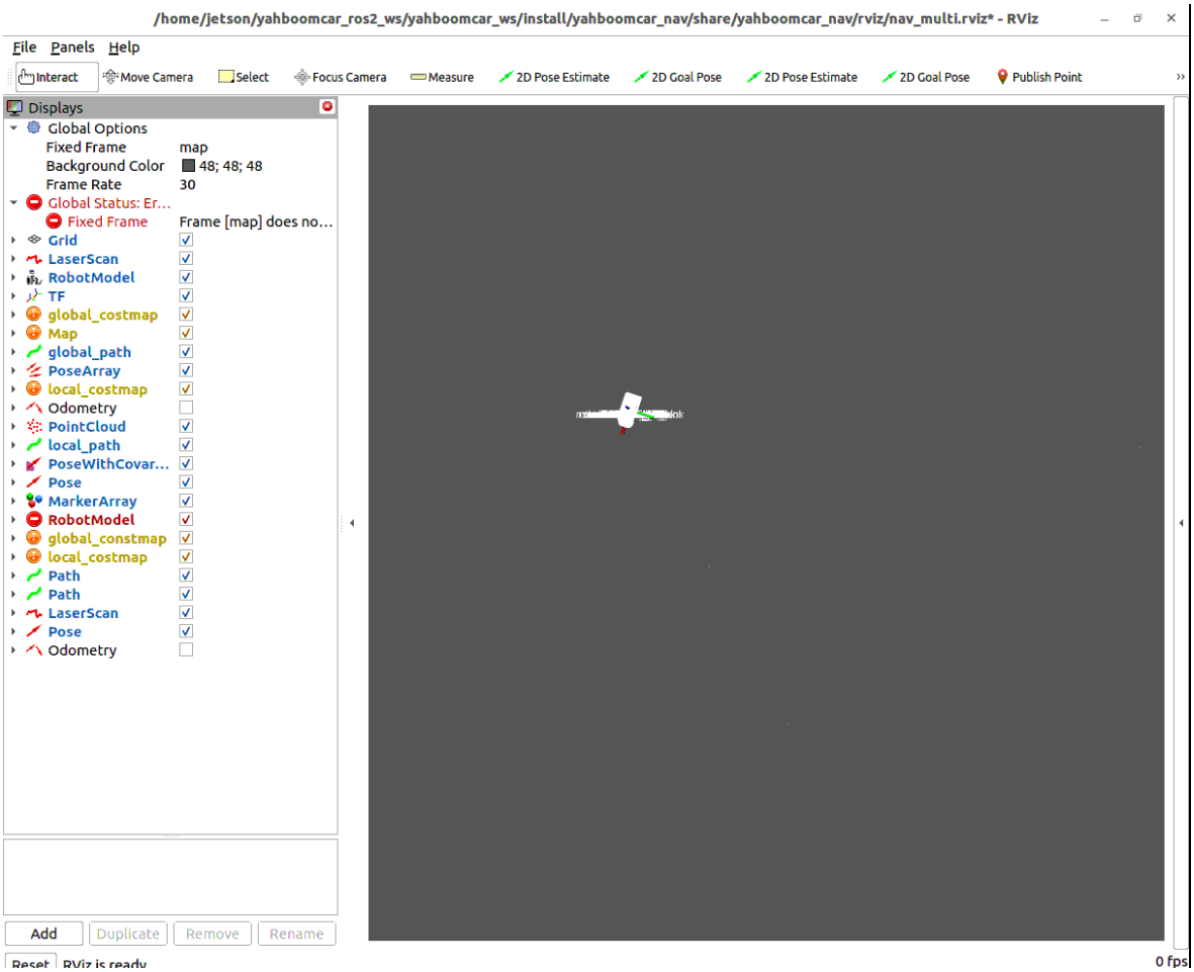
> [!TIP]
> `A1p_laser_bringup_multi_launch.xml` automatically detects the environment variable `RPLIDAR_TYPE` (supporting the `tmini` or `c1` LiDAR) internally, and completes the chassis driver, IMU attitude conversion, EKF filtering, and namespace coordinate frame transformation.

2.2 Step 2: Start RViz2 Visualization and the Global Map Service

Enter the following command in the terminal of the **virtual machine / main board** to start the pre-configured multi-vehicle navigation RViz interface:

```
ros2 launch yahboomcar_multi display_multi_nav_launch.py
```

The interface after startup is shown below:



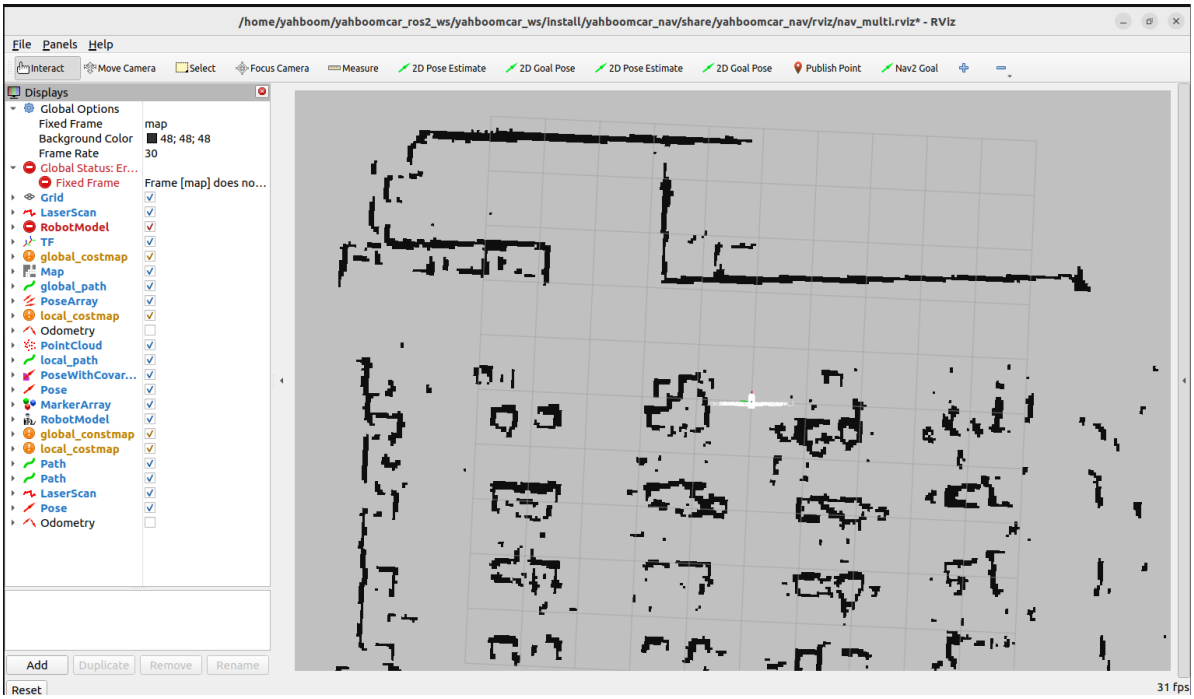
Open another terminal on the **virtual machine / control computer** and start the global map loading service:

```
ros2 launch yahboomcar_multi map_server_launch.py
```

The terminal prints the lifecycle activation status:

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_multi map_server_launch.py
[lifecycle_manager-2] [INFO] [1788598800.374670504] [map_lifecycle_manager]: Creating and initializing lifecycle service clients
[map_server-1] [INFO] [1788598800.410705699] [map_server_node]:
[map_server-1]  map_server_node lifecycle node launched.
[map_server-1]  Waiting on external lifecycle transitions to activate
[map_server-1]  See https://design.ros2.org/articles/node_lifecycle.html for more information.
[map_server-1] [INFO] [1788598800.412357704] [map_server_node]: Creating
[lifecycle_manager-2] [INFO] [1788598800.428062274] [map_lifecycle_manager]: Starting managed nodes bringup...
[lifecycle_manager-2] [INFO] [1788598800.428216326] [map_lifecycle_manager]: Configuring map_server_node
[map_server-1] [INFO] [1788598800.442307068] [map_server_node]: Configuring
[map_server-1] [INFO] [1788598800.442513185] [map_io]: Loading yaml file: /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
[map_server-1] [INFO] [1788598800.473627311] [map_io]: resolution: 0.05
[map_server-1] [INFO] [1788598800.473705457] [map_io]: origin[0]: -9.01
[map_server-1] [INFO] [1788598800.473716817] [map_io]: origin[1]: -15.9
[map_server-1] [INFO] [1788598800.473723409] [map_io]: origin[2]: 0
[map_server-1] [INFO] [1788598800.473729649] [map_io]: free_thresh: 0.196
[map_server-1] [INFO] [1788598800.473735570] [map_io]: occupied_thresh: 0.65
[map_server-1] [INFO] [1788598800.473744210] [map_io]: mode: trinary
[map_server-1] [INFO] [1788598800.473753522] [map_io]: negate: 0
[map_server-1] [INFO] [1788598800.482336911] [map_io]: Loading image_file: /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pgm
[map_server-1] [INFO] [1788598800.544691887] [map_io]: Read map /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pgm: 405 X 518 map @ 0.
5 m/cell
[lifecycle_manager-2] [INFO] [1788598800.554996626] [map_lifecycle_manager]: Activating map_server_node
[map_server-1] [INFO] [1788598800.556942941] [map_server_node]: Activating
[map_server-1] [INFO] [1788598800.558649539] [map_server_node]: Creating bond (map_server_node) to lifecycle manager.
[lifecycle_manager-2] [INFO] [1788598800.672728375] [map_lifecycle_manager]: Server map_server_node connected with bond.
[lifecycle_manager-2] [INFO] [1788598800.672865594] [map_lifecycle_manager]: Managed nodes are active
[lifecycle_manager-2] [INFO] [1788598800.672895419] [map_lifecycle_manager]: Creating bond timer...
```

The loaded 2D grid map will then be displayed normally in the RViz interface:



> [!NOTE]
> If the map does not appear in RViz due to transient packet loss on the LAN, you can press `Ctrl + C` in the map service terminal and re-run `ros2 launch yahboomcar_multi map_server_launch.py`.

2.3 Step 3: Start AMCL Monte Carlo Localization for Both Vehicles

Open another terminal on the control computer and start the localization node of **robot1**:

```
ros2 launch yahboomcar_multi A1p_amcl_robot1_launch.py
```

The terminal prints the node activation status:

```
jetson@yahboom:~$ ros2 launch yahboomcar_multi A1p_amcl_robot1_launch.py
[INFO] [static_transform_publisher-3]: process started with pid [122276]
[static_transform_publisher-3] [WARN] [1788598961.406966805] []: Old-style arguments are deprecated; see --help for new-style arguments
[amcl-1] [INFO] [1788598961.463619423] [robot1_amcl]:
[amcl-1]        robot1_amcl lifecycle node launched.
[amcl-1]        Waiting on external lifecycle transitions to activate
[amcl-1]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[amcl-1] [INFO] [1788598961.466103350] [robot1_amcl]: Creating
[static_transform_publisher-3] [INFO] [1788598961.485619719] [robot1.static_transform_publisher_1sCQ6wWO8aySs1Yb]: Spinning until stopped
- publishing transform
[static_transform_publisher-3] translation: ('0.006058', '0.000000', '0.149120')
[static_transform_publisher-3] rotation: ('0.000046', '0.000046', '-1.000000', '0.000000')
[static_transform_publisher-3] from 'robot1/base_link' to 'robot1/laser'
[lifecycle_manager-2] [INFO] [1788598961.508069178] [robot1_amcl_lifecycle_manager]: Creating
[lifecycle_manager-2] [INFO] [1788598961.517794578] [robot1_amcl_lifecycle_manager]: Creating and initializing lifecycle service clients
[lifecycle_manager-2] [INFO] [1788598961.560820301] [robot1_amcl_lifecycle_manager]: Starting managed nodes bringup...
[lifecycle_manager-2] [INFO] [1788598961.562222412] [robot1_amcl_lifecycle_manager]: Configuring robot1_amcl
[amcl-1] [INFO] [1788598961.563317956] [robot1_amcl]: Configuring
[amcl-1] [INFO] [1788598961.563519497] [robot1_amcl]: initTransforms
[amcl-1] [INFO] [1788598961.593946476] [robot1_amcl]: initPubSub
[amcl-1] [INFO] [1788598961.624652118] [robot1_amcl]: Subscribed to map topic.
[amcl-1] [INFO] [1788598961.653486614] [robot1_amcl]: Received a 405 X 518 map @ 0.050 m/pix
[lifecycle_manager-2] [INFO] [1788598961.661137088] [robot1_amcl_lifecycle_manager]: Activating robot1_amcl
[amcl-1] [INFO] [1788598961.661615658] [robot1_amcl]: Activating
[amcl-1] [INFO] [1788598961.661705548] [robot1_amcl]: Creating bond (robot1_amcl) to lifecycle manager.
[lifecycle_manager-2] [INFO] [1788598961.776756710] [robot1_amcl_lifecycle_manager]: Server robot1_amcl connected with bond.
[lifecycle_manager-2] [INFO] [1788598961.776889705] [robot1_amcl_lifecycle_manager]: Managed nodes are active
[lifecycle_manager-2] [INFO] [1788598961.776913322] [robot1_amcl_lifecycle_manager]: Creating bond timer...
```

Open another terminal and start the localization node of **robot2**:

```
ros2 launch yahboomcar_multi A1p_amcl_robot2_launch.py
```

The terminal prints the node activation status:

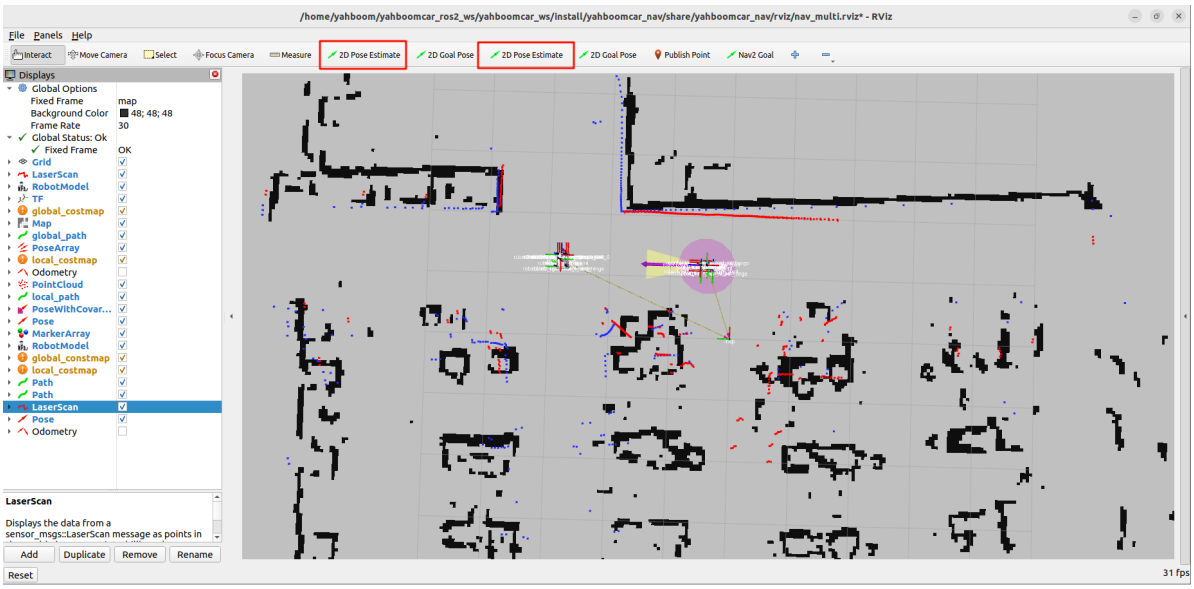
```
jetson@yahboom:~$ ros2 launch yahboomcar_multi A1p_amcl_robot2_launch.py
[lifecycle_manager-2] [INFO] [1788598839.110342183] [robot2_amcl_lifecycle_manager]: Starting managed nodes bringup...
[lifecycle_manager-2] [INFO] [1788598839.110474858] [robot2_amcl_lifecycle_manager]: Configuring robot2_amcl
[amcl-1] [INFO] [1788598839.114304894] [robot2_amcl]: Configuring
[amcl-1] [INFO] [1788598839.114519107] [robot2_amcl]: initTransforms
[amcl-1] [INFO] [1788598839.138233135] [robot2_amcl]: initPubSub
[amcl-1] [INFO] [1788598839.147229270] [robot2_amcl]: Subscribed to map topic.
[amcl-1] [INFO] [1788598839.158498703] [robot2_amcl]: Received a 405 X 518 map @ 0.050 m/pix
[lifecycle_manager-2] [INFO] [1788598839.162080414] [robot2_amcl_lifecycle_manager]: Activating robot2_amcl
[amcl-1] [INFO] [1788598839.162474471] [robot2_amcl]: Activating
[amcl-1] [INFO] [1788598839.162571145] [robot2_amcl]: Creating bond (robot2_amcl) to lifecycle manager.
[lifecycle_manager-2] [INFO] [1788598839.269637127] [robot2_amcl_lifecycle_manager]: Server robot2_amcl connected with bond.
[lifecycle_manager-2] [INFO] [1788598839.269778378] [robot2_amcl_lifecycle_manager]: Managed nodes are active
[lifecycle_manager-2] [INFO] [1788598839.269803147] [robot2_amcl_lifecycle_manager]: Creating bond timer...
[amcl-1] [INFO] [1788598839.283006319] [robot2_amcl]: createLaserObject
[amcl-1] [INFO] [1788598840.183381354] [robot2_amcl]: Message Filter dropping message: frame 'robot2/laser' at time 1788598839.081 for rea
son 'the timestamp on the message is earlier than all the data in the transform cache'
[amcl-1] [WARN] [1788598841.186425155] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598843.190083490] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598845.194441621] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598847.199397017] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598849.202303283] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598851.205658491] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [INFO] [1788598851.352891251] [robot2_amcl]: initialPoseReceived
[amcl-1] [WARN] [1788598851.353216058] [robot2_amcl]: Failed to transform initial pose in time (Lookup would require extrapolation into th
e future.  Requested time 1788598851.353037 but the latest data is at time 1788598851.266526, when looking up transform from frame [robot2
/base_footprint] to frame [robot2/odom])
[amcl-1] [INFO] [1788598851.353266139] [robot2_amcl]: Setting pose (1788598851.353264): 0.175 -0.422 1.042
```

After startup, the terminal prompts you to wait for the initial pose: `AMCL cannot publish a pose or update the transform. Please set the initial pose...`:

```
[amcl-1] [WARN] [1788598841.186425155] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598843.190083490] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598845.194441621] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598847.199397017] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598849.202303283] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
[amcl-1] [WARN] [1788598851.205658491] [robot2_amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
```

Set Initial Poses for Both Vehicles:

1. Observe the top toolbar of RViz, where two dedicated **【2D Pose Estimate】tools** are configured:
   - The first tool is remapped to `/robot1/initialpose`;
   - The second tool is remapped to `/robot2/initialpose`.
2. First click the first pose tool and drag the green orientation arrow on the map according to the actual position of Vehicle 1;
3. Then click the second pose tool and calibrate the orientation on the map according to the actual position of Vehicle 2;
4. After calibration, the LiDAR scan point clouds will perfectly coincide with the black obstacles on the map (red point cloud is robot1's LiDAR data, blue point cloud is robot2's LiDAR data), as shown below:



2.4 Step 4: Start the Nav2 Navigation System for Both Vehicles

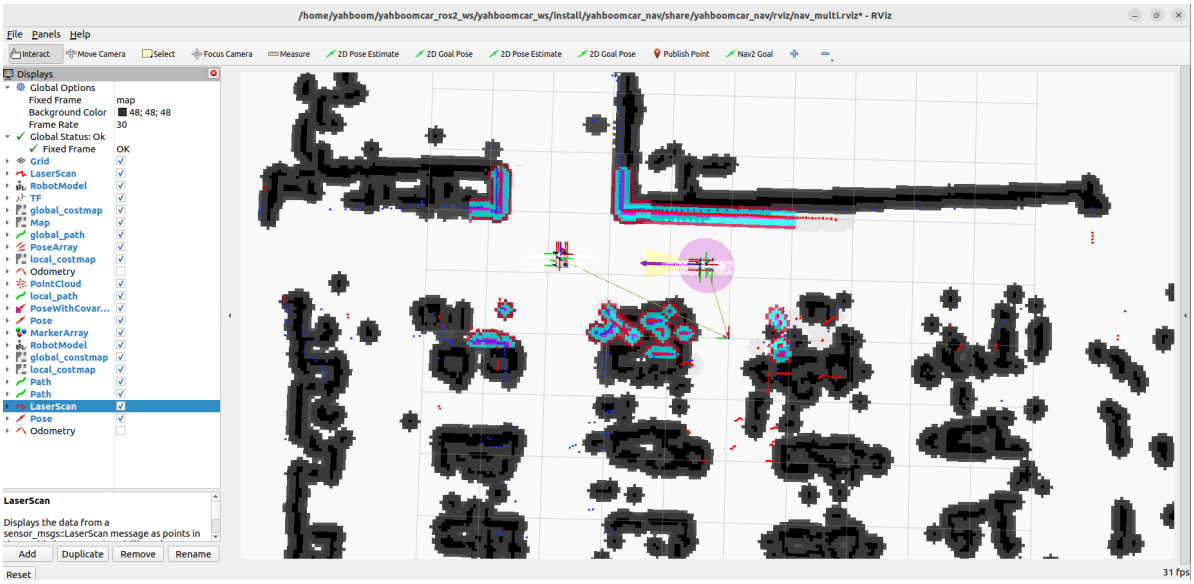
Open another terminal on the control computer and start the Nav2 navigation stack of **robot1**:

```
ros2 launch yahboomcar_multi A1p_nav_robot1_launch.py
```

Open another terminal and start the Nav2 navigation stack of **robot2**:

```
ros2 launch yahboomcar_multi A1p_nav_robot2_launch.py
```

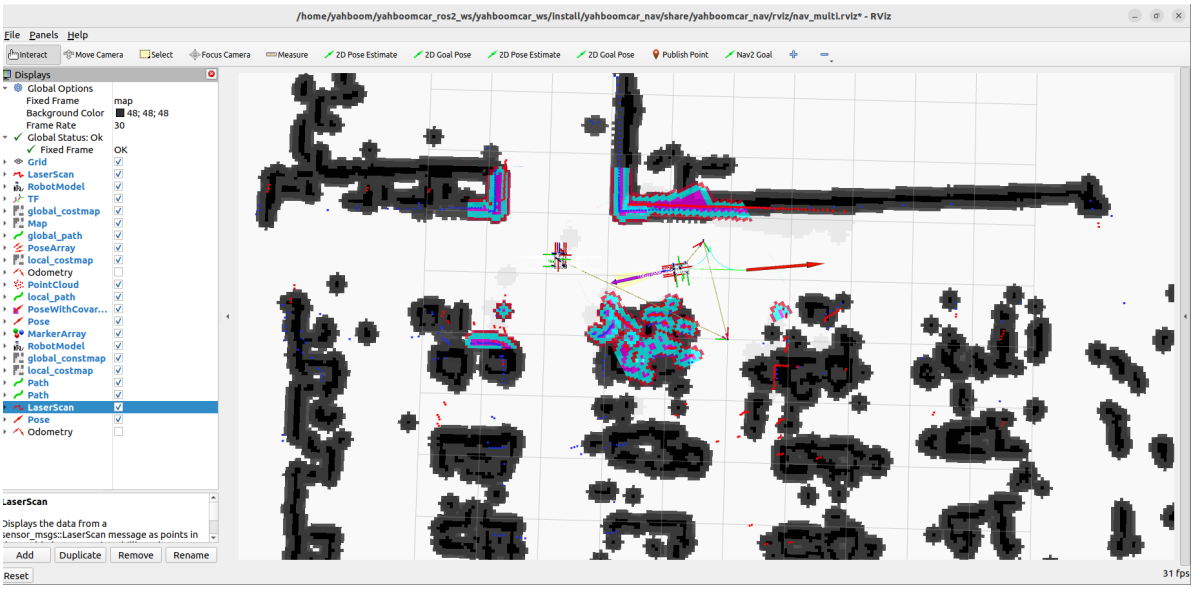
After startup, the costmap inflation layers of both vehicles are rendered and displayed in RViz:



2.5 Step 5: Publish Goal Points for Independent Multi-Vehicle Navigation

Two dedicated goal tools are provided on the RViz top toolbar:

- **The first 【2D Goal Pose】tool**: Sends goal points to `/robot1/goal_pose`;
- **The second 【2D Goal Pose】tool**: Sends goal points to `/robot2/goal_pose`.



Use the corresponding tool for each vehicle to mark goal points on the map. The two vehicles will compute globally optimal paths in parallel and drive along their respective planned routes while avoiding obstacles, safely reaching their own destinations:



3. Core Configuration and Code Analysis

3.1 Chassis and LiDAR Fusion Launch File (A1p_laser_bringup_multi_launch.xml)

Source code path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/A1p_laser_bringup_multi_launch.xml`

```
<launch>
  <arg name="robot_name" default="robot1"/>
  <!-- Automatically obtain the LiDAR model environment variable, default is
tmini -->
  <let name="lidar_type" value="$(env RPLIDAR_TYPE tmini)"/>

  <!-- Include the chassis driver dedicated to the current vehicle -->
  <include file="$(find-pkg-share
yahboomcar_multi)/launch/A1p_bringup_multi.launch.xml">
    <arg name="robot_name" value="$(var robot_name)"/>
  </include>

  <!-- Include the namespaced LiDAR driver according to the model -->
  <group if="$(eval '\'$(var lidar_type)\' == \'tmini\'')">
    <include file="$(find-pkg-share
yahboomcar_multi)/launch/tmini_multi.launch.xml">
      <arg name="robot_name" value="$(var robot_name)"/>
    </include>
  </group>

  <group if="$(eval '\'$(var lidar_type)\' == \'c1\'')">
    <include file="$(find-pkg-share
yahboomcar_multi)/launch/c1_multi.launch.xml">
      <arg name="robot_name" value="$(var robot_name)"/>
    </include>
  </group>
```

```
</launch>
```

- This file encapsulates the underlying hardware driver startup logic. When calling externally, only `robot_name` needs to be passed to automatically bind the LiDAR data and chassis coordinate frames to the namespace tree of the vehicle (e.g., `robot1/scan`, `robot1/odom`).

3.2 Global Map Service Management (map_server_launch.py)

Source code path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/map_server_launch.py`

```
def generate_launch_description():
    map_file = LaunchConfiguration(
        'map',

 default='/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps
/yahboomcar.yaml'
    )
    # 1. Map server node
    map_node = Node(
        name="map_server_node",
        package='nav2_map_server',
        executable='map_server',
        parameters=[{'use_sim_time': False}, {'yaml_filename': map_file}],
        output="screen"
    )
    # 2. Lifecycle manager: automatically configure and switch map_server to the
Active state
    life_node = Node(
        name="map_lifecycle_manager",
        package='nav2_lifecycle_manager',
        executable='lifecycle_manager',
        output='screen',
        parameters=[{'use_sim_time': False}, {'autostart': True}, {'node_names':
['map_server_node']}]
    )
    return LaunchDescription([map_node, life_node])
```

- The map service runs independently of the robot bodies in the root scope without a namespace, publishing the global `/map` topic, so all robots are aligned to a unified global coordinate origin.

3.3 Independent AMCL Launch and Pose Remapping (A1p_amcl_robot1_launch.py)

Source code path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/A1p_amcl_robot1_laun
ch.py`

```
def generate_launch_description():
    param_file = os.path.join(get_package_share_directory('yahboomcar_multi'),
'param', 'A1p_amcl_robot1.yaml')
    amcl_node = Node(
        name="robot1_amcl",
        package='nav2_amcl',
```

```
        executable='amcl',
        parameters=[param_file],
        # Remap the default /initialpose to the dedicated initial pose topic of
the current vehicle
        remappings=[('/initialpose', '/robot1/initialpose')],
        output="screen"
    )
    life_node = Node(
        name="robot1_amcl_lifecycle_manager",
        package='nav2_lifecycle_manager',
        executable='lifecycle_manager',
        output='screen',
        parameters=[{'use_sim_time': False}, {'autostart': True}, {'node_names':
['robot1_amcl']}]
    )
    # Publish the static transform of the vehicle LiDAR base
    tf_base_link_to_laser = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        namespace="robot1",
        arguments=['0.0060585', '0', '0.14912', '0', '3.1415', '3.1415',
'robot1/base_link', 'robot1/laser']
    )
    return LaunchDescription([amcl_node, life_node, tf_base_link_to_laser])
```

- Through the remapping `('/initialpose', '/robot1/initialpose')`, the interactive tools in RViz can precisely distinguish the two vehicles and initialize the particle filter convergence for each of them separately.

3.4 Dedicated Nav2 Navigation Pipeline Integration (A1p_nav_robot1_launch.py)

Source code path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_multi/launch/A1p_nav_robot1_launch.py`

- It introduces `bringup_launch.py` and sets `namespace` to `robot1`;
- It loads `yahboomcar_multi/param/A1p_nav_robot1.yaml`, which strictly configures `robot_base_frame: robot1/base_footprint`, `global_frame: map`, and the subscribed topic paths for both the local and global costmaps;
- The planner and controller servers of the two vehicles operate independently of each other, completely eliminating control command cross-talk.